

I. MỘT SỐ DẠNG BÀI TOÁN QUY HOẠCH ĐỘNG

(DP - Dynamic Programming)

1. Tìm dãy các phần tử là dãy con dài nhất thoả mãn điều kiện bài toán (LIS – Longest Increasing Subsequence)

Phương pháp chung:

- Hàm mục tiêu f là độ dài dãy con
- Sử dụng mảng một chiều L để lưu kết quả với $L[i]$ là độ dài dãy con dài nhất thoả mãn điều kiện bài toán với các phần tử lấy trong miền $a_1, a_2, \dots, a_i$

$$L[1]=1$$

$$L[i] = \max\{1, L[j] + 1 \text{ với } \forall j : 0 < j < i \text{ và } a_j, a_i \text{ thoả mãn điều kiện bài toán}\}$$

Khi đó độ dài dãy con dài nhất là $\max(L[i])$

- Sử dụng mảng một chiều T để lưu vết các phần tử được chọn với $T[i]=j$ có nghĩa j là chỉ số phần tử đứng ngay trước a_i trong dãy kết quả.

Quy trình truy vết mảng T (dùng mảng kq lưu kết quả):

Bài 1. Dãy con đơn điệu không giảm dài nhất

Cho dãy số $a_1, a_2, \dots, a_n$. Hãy tìm một dãy con đơn điệu không giảm có nhiều phần tử nhất của dãy.

Dữ liệu vào: Tập Daycon.inp gồm 2 dòng

- Dòng 1: Số nguyên dương n

- Dòng 2: Gồm n số thực

Dữ liệu ra: Ghi vào tệp Daycon.out

- Dòng 1: Độ dài dãy con tìm được

- Dòng 2: Dãy con đơn điệu không giảm dài nhất

*** Thuật toán:**

Bài toán chính là tìm dãy con đơn điệu dài nhất thoả mãn điều kiện với $\forall j : 0 < j < i$ và $a_j \leq a_i$

daycon.inp
11
5 2 3 4 9 10 5 6 7 8 1
daycon.out
7
2 3 4 5 6 7 8

Bài 2. Bố trí phòng họp

Có n cuộc họp, cuộc họp thứ i bắt đầu từ thời điểm a_i và kết thúc vào thời điểm b_i . Do chỉ có một phòng họp hội thảo nên 2 cuộc họp bất kì sẽ được cùng bố trí phục vụ nếu khoảng làm việc của chúng chỉ giao nhau tại đầu mút. Hãy bố trí phòng họp để phục vụ được nhiều cuộc họp nhất.

Dữ liệu vào: Tập Hop.inp gồm

- Dòng 1: Số nguyên dương n là số cuộc họp
- n dòng tiếp theo mỗi dòng gồm 2 số chỉ thời điểm bắt đầu a_i và kết thúc vào thời điểm b_i của cuộc họp thứ i

Dữ liệu ra: Ghi vào tập Hop.out

- Dòng 1: Số cuộc họp được bố trí
- Các dòng tiếp theo mỗi dòng gồm 2 số chỉ thời điểm bắt đầu và kết thúc của các cuộc họp được bố trí

Hop.inp	Hop.out
5	3
1 3	1 2
1 2	3 4
3 4	5 6
2 5	
5 6	

*** Thuật toán:**

- Sắp xếp các cuộc họp tăng dần theo thời điểm kết thúc (b_i). Lúc này cuộc họp i sẽ được bố trí sau cuộc họp j khi $j < i$ và $b_j = a_i$ (1)
- Bài toán trở thành tìm dãy các cuộc họp dài nhất thoả mãn điều kiện (1)
- Lưu thông tin các cuộc họp vào mảng một chiều a kiểu bản ghi ($a[i].x, a[i].y$ lần lượt là thời điểm bắt đầu và kết thúc của cuộc họp thứ i)

Bài 3. Cho thuê máy

Trung tâm tính toán hiệu năng có một máy cho thuê nhận được đơn đặt hàng của n khách hàng. Khách hàng i muốn sử dụng máy trong khoảng thời gian a_i đến b_i và trả tiền thuê là c_i . Hãy bố trí lịch thuê máy để tổng số tiền thu được là lớn nhất mà thời gian sử dụng máy của 2 khách hàng bất kì được phục vụ đều không giao nhau.

Dữ liệu vào: Tập may.inp gồm

- Dòng 1: Số nguyên dương n là số khách đặt hàng

May.inp	may.out
5	37
1 3 7	1 2 8
1 2 8	2 5 18
3 4 9	5 6 11
2 5 18	
5 6 11	

- n dòng tiếp theo mỗi dòng gồm 3 số chỉ thời điểm bắt đầu a_i , kết thúc vào thời điểm b_i và trả tiền thuê c_i của khách hàng thứ i

Dữ liệu ra: Ghi vào tệp May.out

- Dòng 1: Số tiền thu được lớn nhất khi cho thuê máy

- Các dòng tiếp theo mỗi dòng gồm 3 số chỉ thời điểm bắt đầu a_i , kết thúc vào thời điểm b_i và trả tiền thuê c_i của khách hàng được cho thuê máy

* **Thuật toán:**

- Sắp xếp các khách hàng tăng dần theo thời điểm kết thúc (b_i).

- Lúc này trở thành tìm dãy con có tổng lớn nhất nhưng có biến thể không phải là số lượng phần tử lớn nhất mà là tổng số tiền thuê lớn nhất.

- Công thức tính $L[i]$ là: $L[i] = \max\{c[i], L[j] + c[i]\}$ với $\forall j : 0 < j < i$ và $b_j = a_i$ }

- Sử dụng mảng **may** một chiều kiểu bản ghi để lưu thông tin ban đầu của mỗi máy (may[i].a, may[i].b, may[i].c lần lượt là số chỉ thời điểm bắt đầu a_i , kết thúc vào thời điểm b_i và trả tiền thuê c_i của khách hàng i)

Cải tiến thuật toán:

Ý tưởng $O(n \log n)$:

- Tạo một vector lis lưu **dãy tăng tạm thời**.
- Duyệt từng phần tử x:
 - o Nếu x lớn hơn phần tử cuối cùng trong lis $\rightarrow$ push_back(x).
 - o Ngược lại $\rightarrow$ tìm vị trí đầu tiên $\geq x$ trong lis bằng lower_bound và thay thế nó bằng x.
- Cuối cùng, độ dài LIS = lis.size().

```
#include <bits/stdc++.h>
using namespace std;
int main() {
    int n;
    cin >> n;
    vector<int> a(n);
    for (int i = 0; i < n; i++) cin >> a[i];

    vector<int> lis; // lưu dãy tăng tạm
    for (int x : a) {
        auto it = lower_bound(lis.begin(), lis.end(), x);
        if (it == lis.end()) lis.push_back(x);
        else *it = x;
    }
    cout << "Kết quả: " << lis.size();
}
```